

SERVAX BLEU:

AQUACULTURE OPERATIONS SUITE

Documentación Técnica de Arquitectura de la Información, Trazabilidad y Wireframes Anotados (Fase 3)

Proyecto: Servax Bleu

Materia: TEDS

Responsable: Fernando Alberto González Borbas (Módulo 05 & 06)

1. Resumen Ejecutivo y Alcance del Proyecto

La plataforma **Servax Bleu** es un sistema integral de gestión acuícola diseñado para centralizar la supervisión de corrales marinos, el monitoreo de telemetría IoT (temperatura, oxígeno disuelto, profundidad y streaming de video), la distribución automatizada de alimento/carnada entre embarcaciones alimentadoras y la proyección predictiva de biomasa mediante inteligencia artificial. El objetivo central es sustituir el manejo manual basado en hojas de cálculo de Excel, eliminando duplicidad de captura y previniendo pérdidas biológicas por contingencias ambientales.

2. Mapeo Completo de Requerimientos y Trazabilidad (RF-01 a RF-21, RNF-01 a RNF-07)

A continuación, se consolida la matriz de trazabilidad que vincula cada **Insight de Origen** derivado de la investigación de usuarios con su correspondiente **Requisito de Sistema** y la **Decisión de Arquitectura** adoptada. La prioridad se calcula dinámicamente mediante la ley embebida de riesgo (factor $1.0 \times$ a $2.5 \times$).

ID	Título	Prio	Insight Origen	Decisión de Arquitectura
RF-01	Módulo de IA Operativa	4.8	INS-01: Administradores pierden horas cruzando datos manuales para predecir pérdidas.	DEC-01: Microservicio Python con Scikit-learn para modelos predictivos.
RF-02	Alertas Predictivas IA	4.5	INS-01: Falta de anticipación ante cambios de temperatura que disparan mortalidad.	DEC-02: Motor de reglas e inferencia estadística en backend Node.js.
RF-03	Gestión de Usuarios y Roles	3.8	INS-02: Riesgo de alteración no autorizada de datos de muestreo.	DEC-03: Esquema de control de acceso basado en roles (RBAC) con JWT.
RF-04	Autenticación Segura	3.5	INS-02: Acceso compartido a computadoras de oficina sin cierre de sesión.	DEC-03: Hashing bcrypt y expiración de tokens JWT a 8 horas.
RF-05	Auditoría de Movimientos	3.2	INS-02: Incapacidad de rastrear quién modificó registros de alimento o peces.	DEC-04: Tabla de logs de auditoría inmutables en PostgreSQL.
RF-06	Carga Masiva (Excel/CSV)	4.9	INS-03: Captura duplicada por dependencia histórica de archivos Excel viejos.	DEC-05: Parser masivo en cliente con SheetJS y validación asíncrona.
RF-07	Validación Automática	4.2	INS-03: Valores atípicos o erróneos capturados por captura rápida en campo.	DEC-06: Esquema de validación estricto en frontend con Yup/Zod.
RF-08	Fórmula de Carnada	4.7	INS-04: Inexactitud y retrasos en la asignación diaria de carnada para barcos.	DEC-07: Algoritmo centralizado en la tabla DISTRIBUCIONALIMENTO.
RF-09	Recálculo Automático	4.3	INS-04: Fallas cuando un barco cambia de capacidad por mantenimiento de emergencia.	DEC-07: Triggers en BD y reactividad en frontend para recálculo instantáneo.

RF-10	Historial de Inventario	4.6	INS-05: Falta de visibilidad unificada sobre condición biológica e hídrica.	DEC-08: Vistas compuestas combinando CORRAL, INVENTARIOPEZ y MUESTREOAGUA.
RF-11	Consulta Comparativa	4.0	INS-05: Imposibilidad de comparar rendimiento de corrales entre temporadas.	DEC-08: Índices optimizados en fechas e IdCorral para respuestas < 500ms.
RF-12	Notificaciones Multicanal	4.8	INS-06: Enterarse tarde de picos de hipoxia u oxígeno bajo en corrales alejados.	DEC-09: Integración de WebSockets y Firebase Cloud Messaging (FCM).
RF-13	Umbrales Configurables	3.9	INS-06: Las condiciones óptimas varían según la temporada y edad del atún.	DEC-09: Almacenamiento de parámetros numéricos en tabla de configuración global.
RF-14	Reportes Operativos	4.4	INS-07: Insumo de tiempo excesivo preparando reportes ejecutivos para gerencia.	DEC-10: Generador de reportes dinámicos con Chart.js en pantalla y PDF.
RF-15	Exportación Multi-formato	4.1	INS-07: Auditores requieren formatos estandarizados (PDF) y procesables (Excel).	DEC-10: Librerías jsPDF y SheetJS integradas en el cliente.
RF-16	Integración Sensores IoT	4.9	INS-08: Lecturas manuales en mar abierto son peligrosas en mal clima y con sesgo.	DEC-11: Protocolo MQTT para recepción de datos telemetry de sensores marinos.
RF-17	Panel de Monitoreo Realtime	4.5	INS-08: Necesidad de una 'torre de control' para ver el estado general de un vistazo.	DEC-11: Componentes reactivos con actualización periódica vía polling / SSE.
RF-18	Historial Crecimiento Anual	4.0	INS-05: Dificultad para calcular el momento óptimo de cosecha de atún.	DEC-08: Tabla estructurada CRECIMIENTOANUAL cruzada con ESPECIE.
RF-19	Interfaz Web Responsiva	4.3	INS-09: El personal se desplaza entre muelles, barcos y oficinas con diferentes equipos.	DEC-12: Diseño Mobile-First utilizando Tailwind CSS.
RF-20	App Móvil Complementaria	4.2	INS-09: Complicado usar laptop pesada en las embarcaciones de alimentación.	DEC-12: PWA (Progressive Web App) con soporte offline y sincronización.
RF-21	Sincronización Offline	4.7	INS-09: Pérdida de señal celular frecuente en zonas de corrales marinos.	DEC-12: Persistencia local con IndexedDB y Service Workers.
RNF-01	Rendimiento (<2.0s)	4.6	INS-10: Lentitud en sistemas anteriores desesperaba a los operadores en prisa.	DEC-13: Caché de consultas pesadas e índices compuestos en base de datos.
RNF-02	Disponibilidad 99.9%	4.8	INS-10: La alimentación del atún no puede detenerse por caídas del sistema.	DEC-14: Infraestructura redundante y balanceo de carga.
RNF-03	Seguridad y Cifrado	4.2	INS-02: Protección de datos comerciales de biomasa y costos de carnada.	DEC-03: Certificados SSL/TLS obligatorios y cifrado AES-256.
RNF-04	Escalabilidad Horizontal	4.0	INS-10: Planes de expansión a nuevas zonas acuícolas el próximo año.	DEC-14: Arquitectura orientada a servicios en contenedores Docker.
RNF-05	Usabilidad y Contraste	4.5	INS-09: Reflejo del sol en pantallas móviles dificulta ver tablas pequeñas.	DEC-12: Modo de alto contraste / Dark mode y botones táctiles amplios.
RNF-06	Interoperabilidad API	4.1	INS-08: Necesidad de conectar sondas de marcas variadas.	DEC-11: Especificación OpenAPI (Swagger) para endpoints públicos.
RNF-07	Respallos Automáticos	4.4	INS-02: Temor a perder el historial de años de producción por fallas de disco.	DEC-13: Script cron para dumps automatizados guardados en la nube.

3. Interfaz 5 — Dinamograma Roper (Segmentación UX)

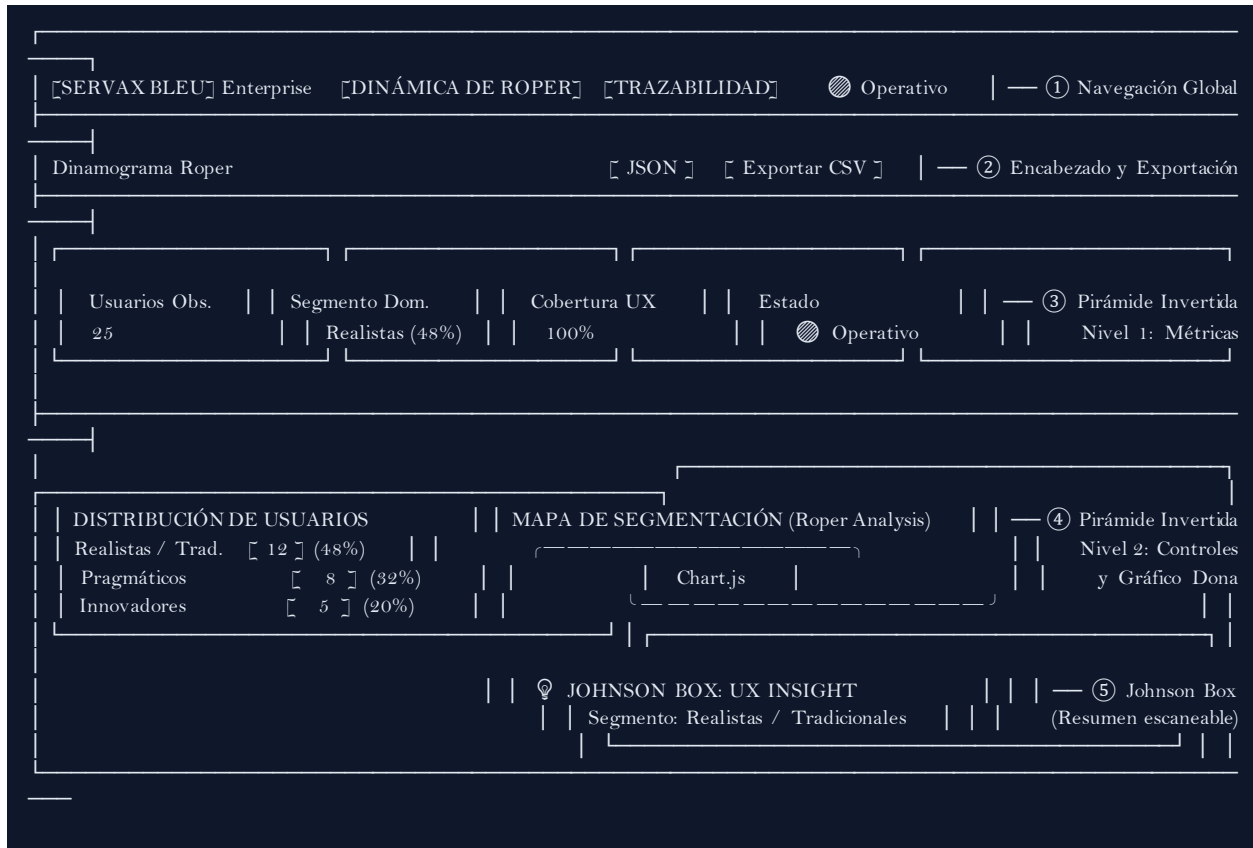
El modelo de Roper segmenta la muestra de usuarios observados en tres grupos clave según sus valores y necesidades operativas:

Segmento	Requisito UX Principal	Funcionalidades Clave	Tono del Sistema
Realistas / Tradicionales (48%)	Control operativo, seguridad estricta y bajo esfuerzo cognitivo.	Autenticación RBAC, bitácoras inmutables e historial por corral.	Formal, confiable y predecible.
Pragmáticos (32%)	Velocidad en captura de datos, carga masiva y tablas de alta densidad.	Carga masiva Excel/CSV, fórmula de carnada y reportes inmediatos.	Directo, técnico y utilitario.
Innovadores (20%)	Dashboards interactivos, analítica con IA y alertas en tiempo real.	Módulo predictivo, telemetría IoT MQTT y notificaciones push.	Moderno, predictivo y tecnológico.

4. Wireframes Anotados de Baja Fidelidad y Algoritmos de Atención (Fase 3)

4.1 Pantalla Clave 1: Interfaz 5 — Dinamograma Roper (#tabInterfaz5)

Esta pantalla corresponde a la vista de segmentación visual y cálculo dinámico de cuotas UX maquetada en el HTML.

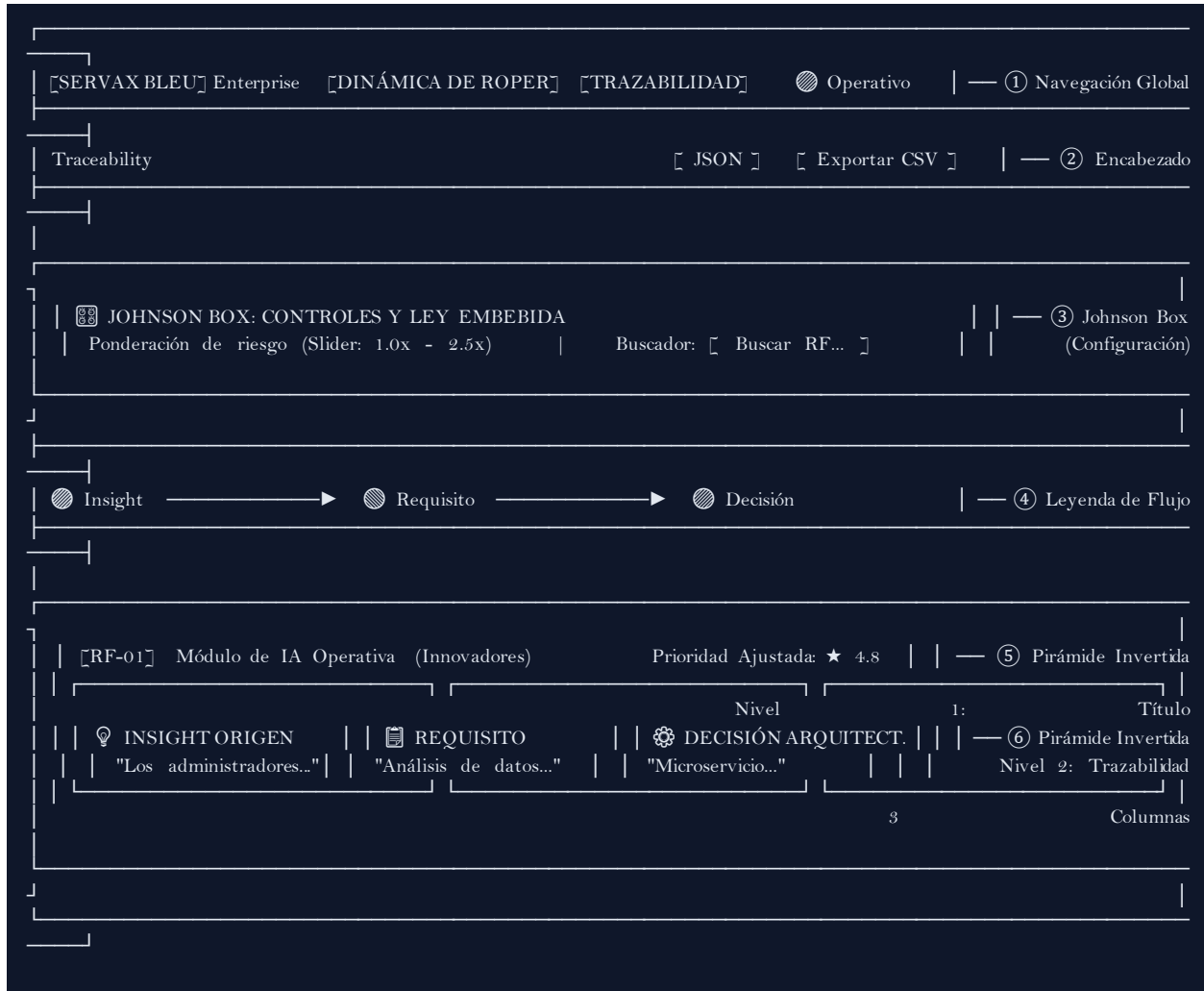


Anotación de las 3 Capas (Interfaz 5)

- **Capa Superior / Interfaz (UI):** Johnson Box (⑤) en el panel #drawerUX que destaca el requisito UX derivado del segmento activo. Pirámide Invertida organizada en Nivel 1 (Métricas generales #metricTotal), Nivel 2 (Inputs de cuotas y canvas de Chart.js) y Nivel 3 (Acciones de exportación JSON/CSV).
- **Capa Media / Task Flow:** (U) Usuario modifica cuota numérica -> (S) Ejecuta recalcularI5() -> (S) Recalcula porcentajes, actualiza gráfico circular y actualiza el panel UX Insight.
- **Capa Base / Base de Datos:** Persistencia de variables de segmentación en app.json y localStorage.

4.2 Pantalla Clave 2: Interfaz 6 — Traceability (#tabInterfaz6)

Esta pantalla corresponde a la vista de trazabilidad en cascada y filtro por ley embebida de riesgo.



Anotación de las 3 Capas (Interfaz 6)

- **Capa Superior / Interfaz (UI):** Johnson Box (③) con slider de ponderación (#sliderPonderacion) y buscador en vivo. Pirámide Invertida en Nivel 1 (Prioridad ajustada y título) y Nivel 2 (Mapeo en 3 columnas: Insight -> Requisito -> Decisión).
- **Capa Media / Task Flow:** (U) Usuario desliza ponderador de riesgo de 1.0x a 2.5x -> (S) Ejecuta recalcularI6() -> (S) Reordena la lista de requerimientos en tiempo real.
- **Capa Base / Base de Datos (ERD_ServaxBleu):** Consultas a las tablas CORRAL, MUESTREOAGUA, DISTRIBUCIONALIMENTO y BARCO.

5. Matriz Diagnóstica (Lógica del Sistema vs. Lógica del Usuario)

Dimensión	Lógica del Sistema (Actual)	Lógica del Usuario (Con AI)
Prioridad	Listado plano de requerimientos sin jerarquía de impacto operativo.	Priorización dinámica por Ley Embebida de Riesgo e impacto en biomasa.
Navegación	Menú estructurado por entidades de BD (Corrales, Barcos, Muestreos).	Navegación por tareas clave (Monitoreo de alertas, Distribución de carnada).
Estructura de Datos	Paginado de tablas crudas sin contexto de origen.	Vista en cascada con trazabilidad directa: Insight -> Requisito -> Decisión.
Medida de Éxito	Consultas ejecutadas en menos de 100 ms.	Cero pérdidas biológicas por hipoxia y 100% de cumplimiento en distribución.

6. Experience Stack (Síntesis de Capas)

CAPA DE ENTREGA (Frontend de la Atención)

- Johnson Box: Banner de alertas de oxígeno/temperatura y resumen de carnada.
- Pirámide Invertida: Jerarquización visual en tarjetas de corrales e Interfaz 6.

CAPA DE ESTRUCTURA (Middleware de Movimiento y AI)

- Trazabilidad: Insight Origen → Requisito Sistema → Decisión Arquitectura.
- Ley Embebida: Slider de ponderación de riesgo (1.0x - 2.5x) con recálculo en vivo.

CAPA DE CIMIENTOS (Backend y Modelo de BD)

- Base de datos relacional de 11 tablas (CORRAL, MUESTREOAGUA, DISTRIBUCIONALIMENTO).
- Captura de sensores IoT MQTT y persistencia en app.json / localStorage.